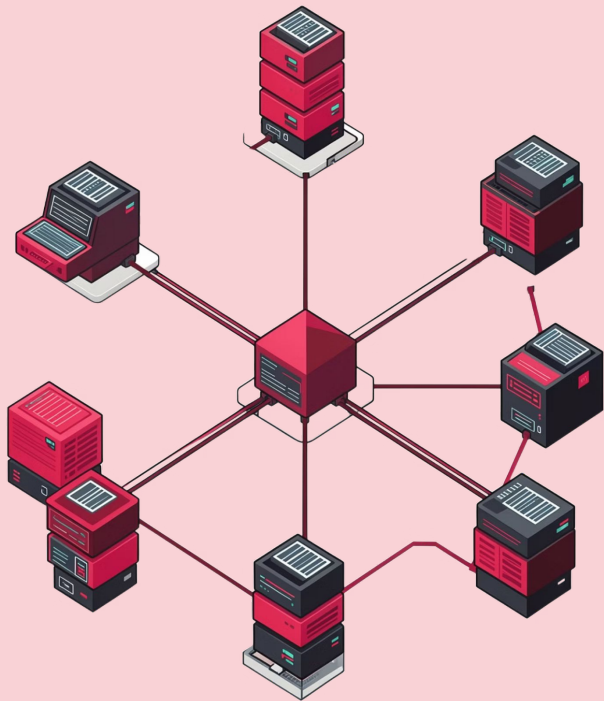




NetFileX: A Distributed Network File System

Built in C on Linux for the Operating Systems and Networks course at IIIT Hyderabad. A fault-tolerant, scalable file system designed to handle concurrent clients and distributed storage across multiple servers.

Team: Shreyas Mehta, Inesh Dheer, Shubham Goel, Swam Singla |

Duration: 3 weeks

Why Not Just One Server?

Traditional single-server architectures present critical limitations for production systems. A single point of failure can bring down the entire system, while scalability bottlenecks prevent handling concurrent clients efficiently.

Single Point of Failure

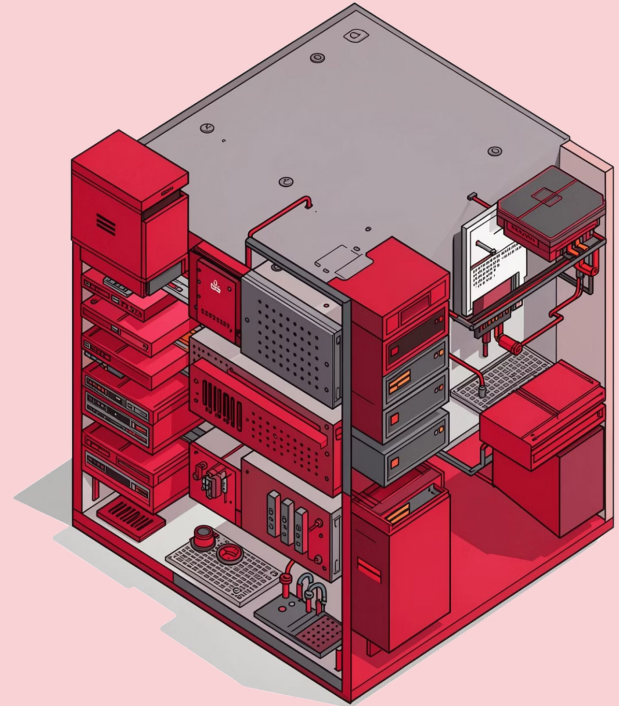
One server crash means complete data unavailability and system downtime

Data Loss Risk

No redundancy means any hardware failure results in permanent data loss

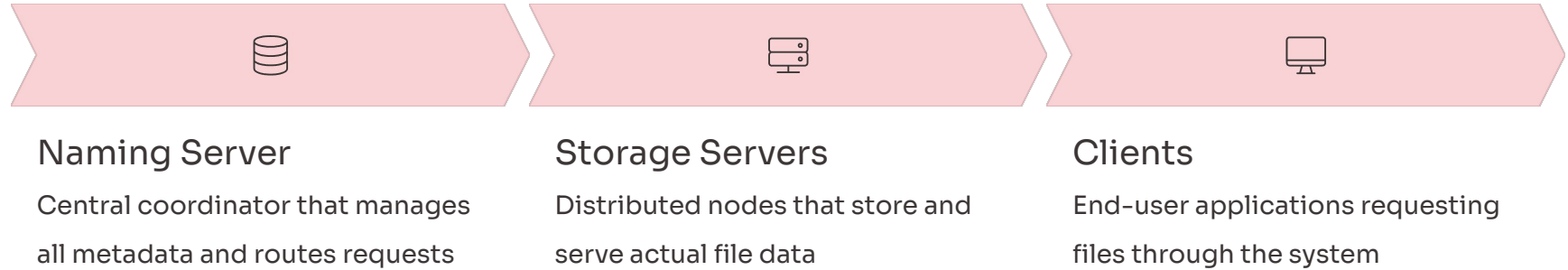
Scalability Bottleneck

Cannot handle many clients simultaneously without performance degradation



SSingulle Serrwone
Failer-:ff7024

Three-Component Architecture



The flow follows a smart routing pattern: **Client** → **Naming Server** → **Storage Server** → **Client**. This separation of concerns ensures efficient metadata management while maintaining data distribution.

Naming Server: The Brain of the System

Central Registry

Maintains complete metadata about all files and their physical locations across storage servers

Intelligent Routing

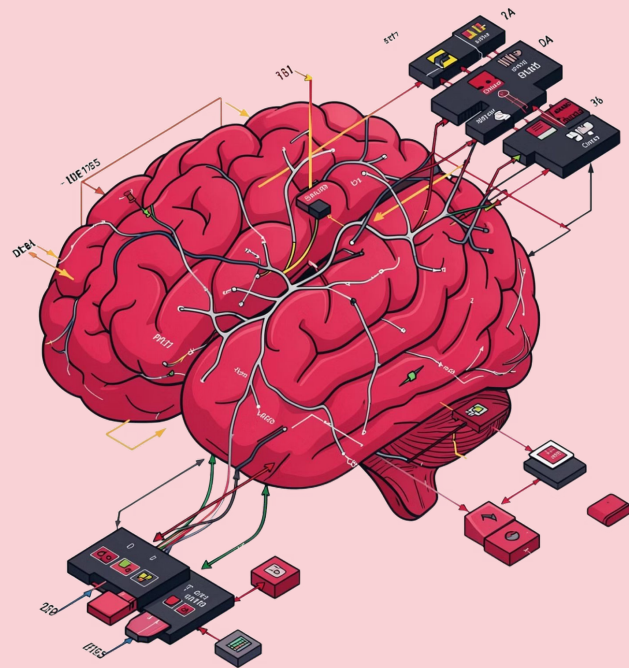
Directs client requests to the appropriate Storage Server based on file location and server health

Fast Path Resolution

Trie-based indexing enables $O(k)$ path lookups where k is path depth, with LRU cache for repeated access

Metadata Only

Never stores actual file data — only tracks where files are located for efficient coordination





Storage Servers: The Distributed Vaults



Data Storage and Retrieval

Store and serve actual file data to authorized clients through efficient read and write operations



Automatic Registration

Self-register with the Naming Server upon startup, creating dynamic infrastructure that adapts to changes



Scalable Design

System supports up to 100 storage servers, allowing horizontal scaling as demand grows

Client Interface: Full File System Commands



LIST

Browse directory contents and file metadata



READ

Retrieve file contents from distributed storage



WRITE

Create or update files across the network



DELETE

Remove files and free storage space



COPY

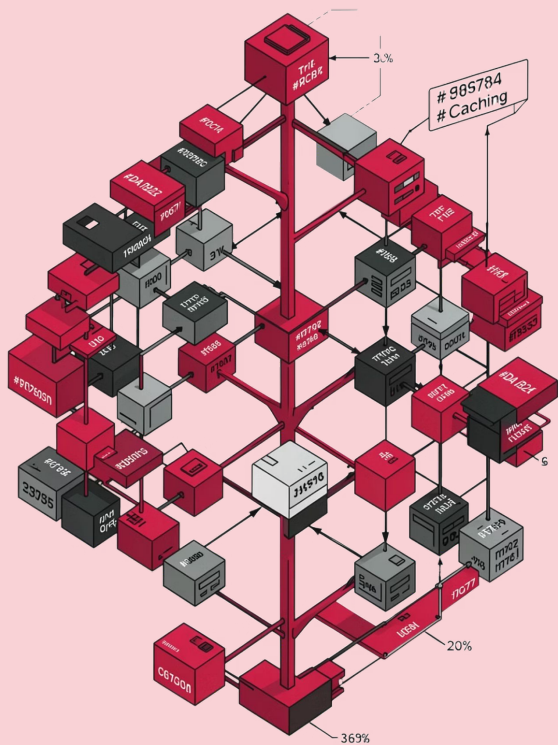
Duplicate files within the distributed system



AUDIO

Real-time MP3 streaming via mpv integration

Clients first contact the Naming Server to resolve file locations, then communicate directly with Storage Servers for optimal performance.



Trie + LRU Cache: Speed by Design

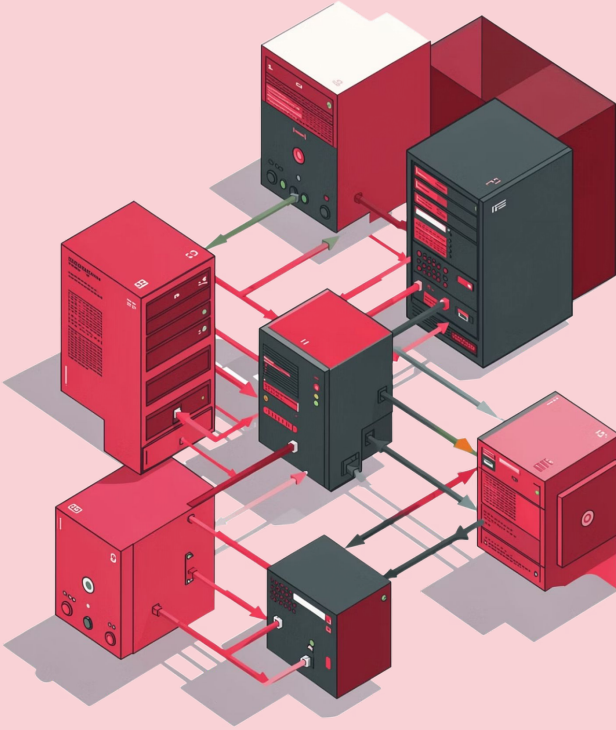
Trie-Based Indexing

Hierarchical tree structure enables **$O(k)$ file path lookups** where k is the path depth. This hierarchical structure perfectly matches file system path semantics for lightning-fast resolution.

Both data structures reside within the Naming Server, working together to deliver sub-millisecond metadata resolution even under heavy load.

LRU Cache Optimization

Recently accessed paths are cached for **$O(1)$ hit/miss detection**. Implemented as doubly-linked list plus hashmap, storing 10 most recent paths for optimal repeated access performance.



Built to Survive: Replication and Failover



Automatic Replication

Every file automatically replicated to SS[n+1] and SS[n+2] servers for redundancy



Heartbeat Monitoring

Continuous PING/PONG protocol monitors server health in real-time



Automatic Failover

On primary failure, system instantly switches to backup server seamlessly



Async Replication

Non-blocking replication ensures client operations complete quickly

Many Readers, One Writer — Simultaneously



Readers-Writers Lock

Multiple concurrent readers can access files simultaneously, while writers get exclusive access for data consistency



Pthread Pool

Each client connection handled by dedicated pthread, enabling true concurrent processing across all components



Async Writes

Files over 8KB use asynchronous writes — client receives ACK instantly while write continues in background



Mutex Protection

Critical sections throughout codebase protected by mutex locks to prevent race conditions and data corruption



What We Built in 3 Weeks

Achievements

- Supports 100 Storage Servers and 100 concurrent clients
- Automatic replication and heartbeat-based failover
- Real-time audio streaming integration
- Complete CRUD file operations over network
- Trie + LRU cache for optimal metadata resolution

Future Enhancements

- End-to-end encryption for secure file transfers
- Horizontal scaling of Naming Server
- Checksum validation for data integrity verification
- Compression for network bandwidth optimization